

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN LẬP TRÌNH PYTHON



Báo cáo bài tập

Họ và tên sinh viên:

Nguyễn Bá Khánh

Mã sinh viên:

B23DCCE051

Lớp:

D23CQCE06-B

Hà Nội – 2025

Bài 1:

- Những tập lệnh được dùng:

Lấy dữ liệu từ trang web:

```
driver1 = webdriver.Firefox()
driver1.get('https://fbref.com/en/comps/9/stats/Premier-League-Stats')
soup = BeautifulSoup(driver1.page_source, 'html.parser')
driver1.quit()
```

Ta sẽ lấy được thông tin trên trang web đó, cụ thể như trên là những thông tin chung của các cầu thủ, và cần chọn đúng vị trí thông tin liên quan:

```
temp1 = soup.find('div', attrs = {'id' : 'all_stats_standard'})
temp2 = temp1.find_all('tr', attrs = {'data-row': True})
```

Trong bảng thông tin đó cần lấy lần lượt theo cầu thủ:

```
for x in range(len(temp2)):
    try:
        time = temp2[x].find('td', attrs = {'data-stat' :
'minutes_90s'}).text.strip()
    except AttributeError:
        continue
```

Sau khi đã lấy được hết thông tin thì xuất ra file CSV:

```
df = pd.DataFrame(danh)
df.to_csv('Stats.csv', index=False)
```

Bài 2:

- Những tập lệnh được dùng:

Lấy thông tin từ file CSV:

```
with open('Stats.csv', mode='r', encoding='utf-8-sig') as file:
    csvFile = csv.reader(file)
    trg = 0
    for lines in csvFile:
        if trg == 0:
            trg += 1
            continue
```

Mỗi vòng lặp sẽ cho thông tin đầy đủ của một cầu thủ, và cần đưa thông tin đó vào list, để có thể sắp xếp lấy ra 3 cầu thủ có chỉ số lớn nhất của lĩnh vực đó:

```
f = open('top_3.txt', 'x')
a.sort(key = lambda x: -float(x.Goals))
with open('top_3.txt', 'a') as s:
    s.write('Highest Games: {}, {}, {}\n'.format(a[0].Name, a[1].Name, a[2].Name))
```

Ví dụ trên sẽ lấy được 3 người chơi nhiều trận nhất, cần làm tương tự với các chỉ số khác. Sau khi ghi vào nhóm chỉ số cuối cùng ta đã có file ghi thông tin của những người có chỉ số cao nhất trong nhóm.

Tiếp theo cần lấy ra số trung vị, số trung bình và độ lệch chuẩn của các chỉ số:

```
numeric_cols = df.select_dtypes(include='number').columns.tolist()

overall_stats = df[numeric_cols].agg(['mean', 'median', 'std'])
overall_stats['Team'] = 'Overall'
overall_stats = overall_stats.set_index('Team')
```

Select_dtypes(include='number').columns.tolist(): Chọn các cột có dữ liệu số và chuyển sang thành một danh sách. Sau đó tính trung vị, trung bình và độ lệch chuẩn cho tất cả các cột số. Sau đó cần tính các chỉ số đó theo nhóm:

```
if 'Team' in df.columns:
    team_stats = df.groupby('Team')[numeric_cols].agg(['mean', 'median', 'std'])

    team_stats.columns = ['_'.join(col).strip() for col in
team_stats.columns.values]

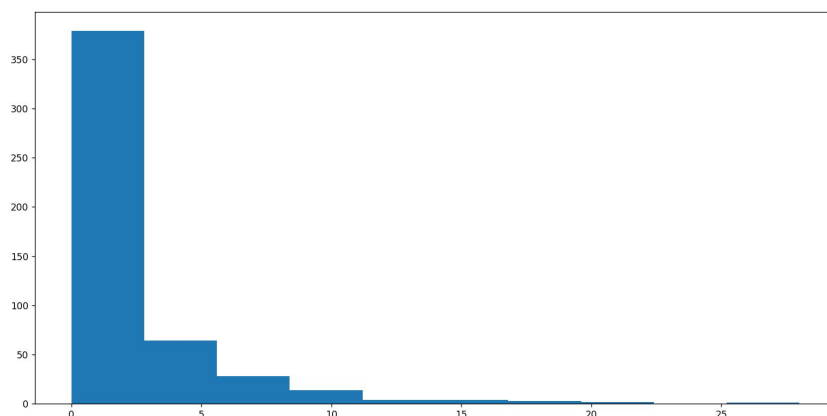
    team_stats.reset_index(inplace=True)

    overall_stats_flat = overall_stats.reset_index()

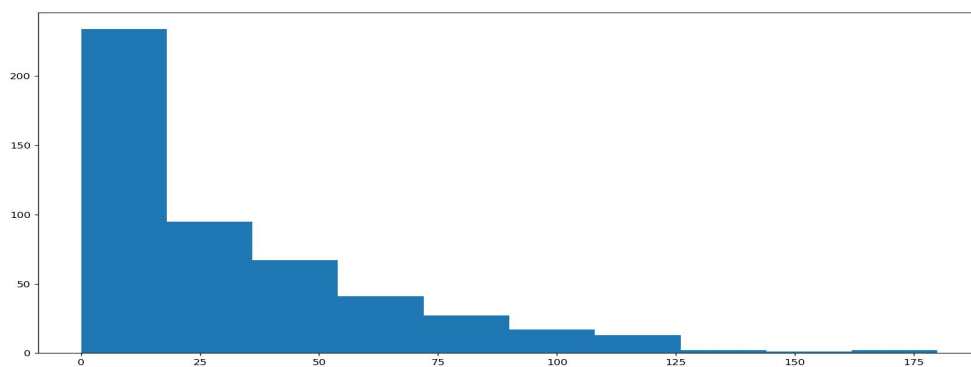
    final_df = pd.concat([overall_stats_flat, team_stats], ignore_index=True)
else:
    final_df = overall_stats.reset_index()
```

Cuối cùng xuất ra một file CSV.

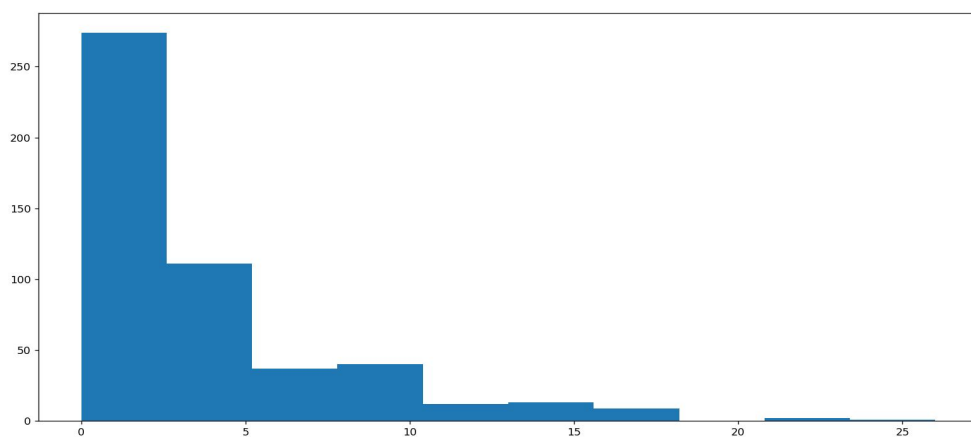
- Một số biểu đồ tần suất:



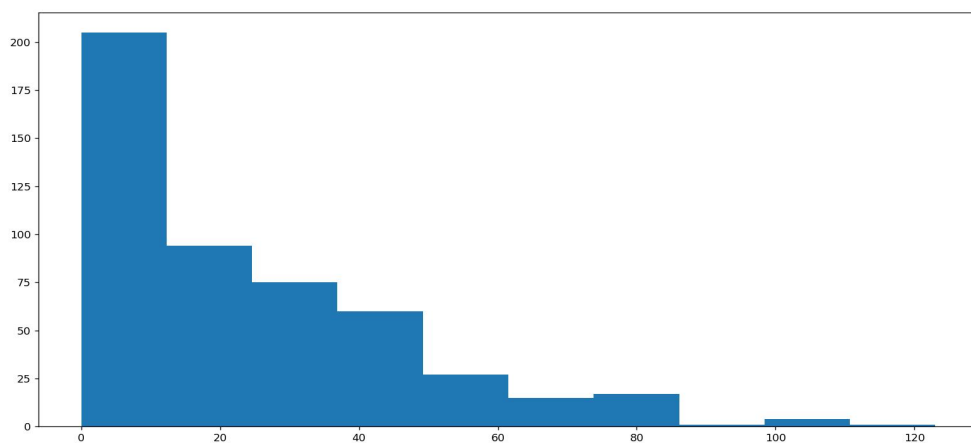
Tần suất ghi bàn



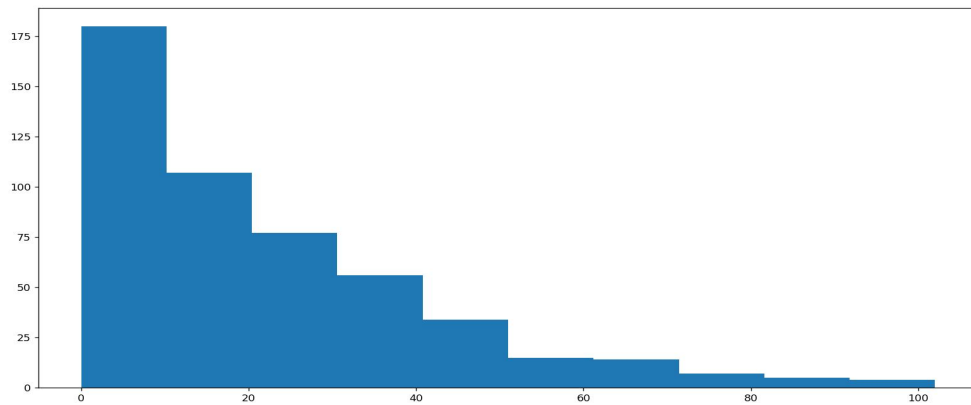
Tần suất kiến tạo



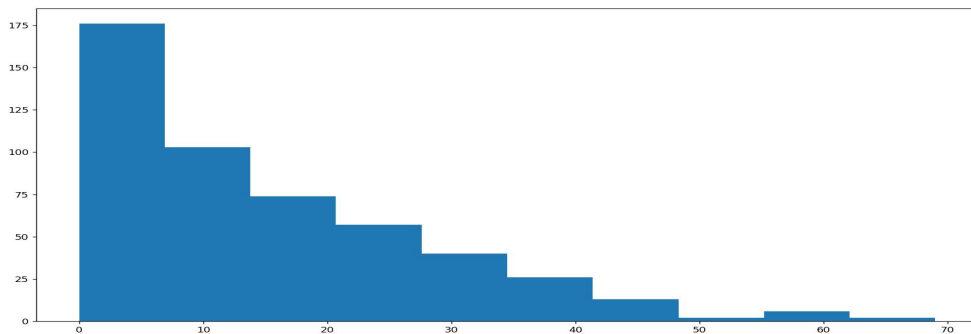
Tần suất goal creation



Tần suất tackle



Tần suất attack



Tần suất block

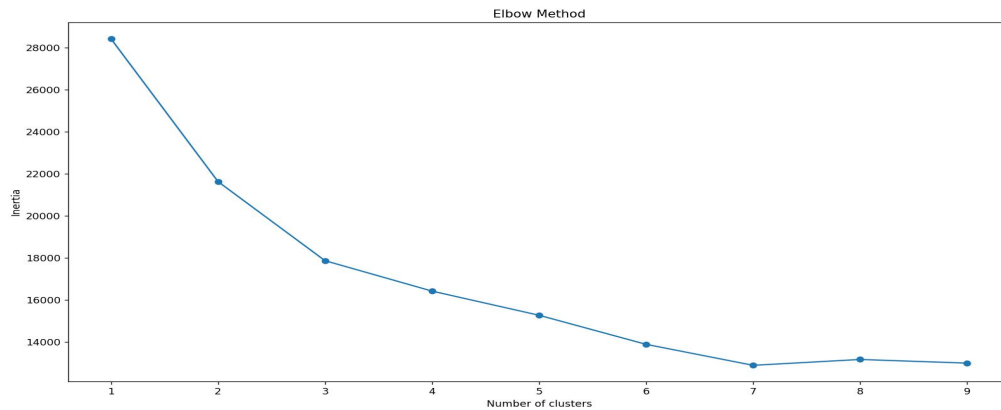
Bài 3:

Trước tiên cần xác định xem cần chia bao nhiêu nhóm. Với phương pháp Elbow có thể xác định số lượng nhóm tối ưu. Phương pháp này dựa trên tổng bình phương khoảng cách từ các điểm đến trung tâm cụm. Cần tìm điểm khuỷu tay mà tăng cụm không làm giảm inertia đáng kể:

```
inertias = []
for k in range(1, 10):
    kmeans = KMeans(n_clusters=k, random_state=0)
    kmeans.fit(data_scaled)
    inertias.append(kmeans.inertia_)

plt.plot(range(1, 10), inertias, marker='o')
plt.xlabel('Number of clusters')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.show()
```

Qua đó sẽ có biểu đồ:



Hàm nhóm theo thuật toán K-means:

```
kmeans = KMeans(n_clusters=7, random_state=0)
```

```
labels = kmeans.fit_predict(data_scaled)
```

```
plt.figure(figsize=(8, 6))
```

```
data_np = np.array(data)
```

```
for i in range(7):
```

```
    cluster = data_np[labels == i]
```

```
    plt.scatter(cluster[:, 0], cluster[:, 1], label=f'Group {i+1}')
```

Kết quả thu được:

